DATA STRUCTURE AND APPLICATIONS
Module 2: Stacks and Queues

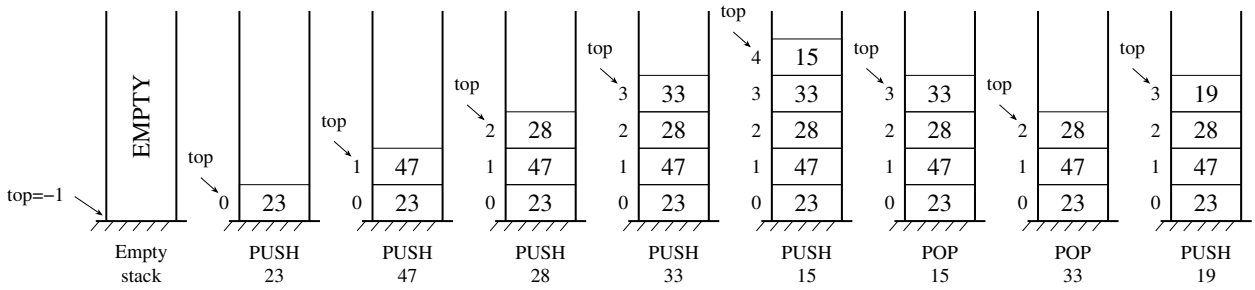
Stacks

A stack is a non-primitive linear data structure, which is an ordered list (Array) in which insertions (also called PUSHes) and deletions (also called POPs) are made at one end called the top.

Given a stack S = (a_0, a_1, . . . , a_{n-1})
where a_0 : is the bottom element in stack.
a_{n-1} : is the top element in stack.

The restrictions on the stack imply that if we push the elements 23, 47, 28, 33 and 15 to the stack, in that order, then 15 is the first element we pop from the stack. Since the last element inserted into a stack is the first element removed, a stack is also known as a Last-In-First-Out (LIFO) array/list.

Below figure illustrates the operations (diff) on stack.



Operations on a stack:

i. PUSH: PUSH operation adds/insert an element to the top of the stack.

ii. POP: POP operation deletes/removes the element from the top of the stack.

iii. PEEK: PEEK operation returns the value of the topmost element of the stack.

iv. DISPLAY: Display operation display the status of stack.

Array representation of Stacks.

The stack can be represented as a linear array. Every stack has a variable called TOP associated with it, which is used to store the address/index of the topmost element of the stack. The TOP is the position where the element will be inserted (PUSH) (or) deleted (POP) from stack. There is another variable MAX, which is used to store the maximum number of elements that the stack can hold.

if TOP = −1, it indicates stack is empty (initial value of top)
if TOP = MAX−1, it indicates stack is full
(MAX−1, it is because array index start from 0).

The easiest way to implement stack using 1-D array:

```
#define MAX 5
int top = -1;
int stack[MAX];
```

Note: Declare top as global variable. This variable will be used in all stack functions.

where: MAX is the maximum no of entries.
stack[0] : First (or) bottom element of stack
stack[1] : Next (or) Second element of stack
⋮
stack[MAX−1] : Last element of stack.
top is set to −1 : To denote an empty stack.
ie initial value of top is −1.

Error Conditions on Stack:

i. Stack Overflow → if top = MAX−1
ii. Stack Underflow → if top = −1
iii. Stack Empty → if top = −1

PUSH Operations:

The PUSH operation is used to insert an element into the stack. The new element is added at the topmost position of the stack. However, before inserting the value, we must first check if top = MAX−1, because if that is the case, then stack is full and no more insertions can be done. If an attempt is made to insert a value in a stack that is already full, an OVERFLOW message is printed.

Overflow Function.

```
int stack_overflow()
{ if (top == MAX-1)
     return 1;
  else
     return 0;
} /* End of overflow */
```

where top is declared as global variable. So no need to pass top parameter to function.

PUSH Function.

```
void push(int stack[], int value)
{
   top = top+1;
   stack[top] = value;
   return;
} /* End of PUSH */
```

(or)
```
stack[++top] = value;
   return;
```

where stack[] is a array and element to be pushed ie value are passed from main() function. and before calling push() function check for overflow().

POP Operation:

The POP operation is used to delete the topmost element from the stack. However, before deleting the value, we must first check if TOP = −1, because if that is the case, then it means the stack is EMPTY and no more deletions can be done. If an attempt is made to delete a value from a stack that is already empty, an UNDERFLOW message is printed.

Underflow Function.

```
int stack_underflow()
{ if (top == -1)
     return 1;
  else
     return 0;
} /* End of Underflow */
```

where top is declared as global variable.

POP Function.

```
int pop(int stack[])
{ int x;
  x = stack[top];
  top = top-1;
  return (x);
} /* End of pop */
```

(or) `return(stack[top-]);`

PEEK Operation:

PEEK is an operation that return the value of the topmost element of the stack without deleting it from the stack. PEEK function also check for stack UNDERFLOW, print appropriate message, else returns the value.

```
int peek(int stack[])
{ int x;
  x = stack[top];
  return (x);
} /* End of PEEK */
```

(or) `return(stack[top]);`

Display Function:

Display function display the status of the stack from topmost element to bottom element. DISPLAY function also check for stack empty using top = −1, print appropriate message “stack is Empty” else returns the content of stack.

Empty Function.

```
int stack_empty()
{ if (top == -1)
     return 1;
  else
     return 0;
} /* End of stack empty */
```

top is declared as global variable

Display Function:

```
void stack_display()
{ int i;
  for(i=top; i>=0; i--)
     printf("%d \n", stack[i]);
  printf("\n");
  return;
} /* End of display */
```

Pgm: Write a 'C' program to perform PUSH, POP, PEEK and Display operations on stack using static array.

```
#include<stdio.h>
#include<stdlib.h>
#define MAX 5
int top=-1;
int main()
{ int stack_overflow();
  int stack_underflow();
  int stack_empty();
  void push(int stack[], int value);
  int pop(int stack[]);
  int peek(int stack[]);
  void stack_display(int stack[]);
  int stack[MAX], value, choice, x;

  do
  { printf("Enter 1-PUSH \n 2-POP \n
           3-PEEK \n 4-DISPLAY \n
           any value for EXIT \n");
    scanf("%d", &choice);
    switch(case)
    { case 1: if (stack_overflow())
                 printf("Stack Overflow \n");
              else
              { printf("Enter the value to be pushed \n");
                scanf("%d", &value);
                push(stack, value);
              }
              break;
      case 2: if (stack_underflow())
                 printf("Stack Underflow \n");
```

```
                else
                { x = pop(stack);
                  printf("POPed element from
                          stack is %d \n", x);
                }
                break;
        case 3: if (stack_underflow())
                   printf("Stack Underflow \n");
                else
                { x = peek(stack);
                  printf("Top most element in
                          stack is %d \n", x);
                }
                break;
        case 4: if (stack_empty())
                   printf("Stack is Empty \n");
                else
                { printf("Stack status \n");
                  stack_display(stack);
                }
                break;
        default: printf("Invalid input - EXIT \n");
                 exit(0);
      } /* End of switch-case */
    } while (choice >=1 && choice <=4); /* End of do-while */
    return 0;
} /* End of Main program */
```

Representation of Stack using Dynamic Array

In a static array the size MAX is fixed at compile time. If the stack becomes full, no more elements can be pushed, even when memory is available. In a dynamic array the memory for the stack is taken at run time using `malloc()`. When the stack becomes full, a bigger block of memory is taken using `realloc()` and the stack continues to grow.

Idea:

- Start with a small `capacity` (say 2).
- On PUSH, if top = capacity−1 (stack full), double the capacity using `realloc()`. The old elements are kept automatically.
- POP, PEEK and DISPLAY work exactly as in the static array.
- At the end, release the memory using `free()`.

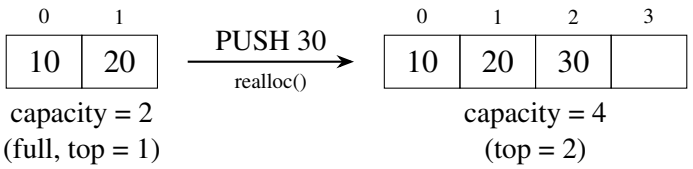


Fig: Stack grows when it is full (dynamic array).

Pgm: Write a C program to implement a stack using dynamic array.

```
/* Stack using a dynamic array: the stack grows when it is full */
#include <stdio.h>
#include <stdlib.h>

int *stack;                   /* memory comes from malloc() */
int top = -1;
int capacity = 2;             /* starting size of the stack */

void push(int value)
{
    if (top == capacity - 1) {          /* stack is full */
        capacity = capacity * 2;        /* double its size */
        stack = (int *)realloc(stack, capacity * sizeof(int));
        if (stack == NULL) {
            printf("Memory not available\n");
            exit(1);
        }
        printf("Stack full: size increased to %d\n", capacity);
    }
    stack[++top] = value;
}

int pop(void)
{
    return stack[top--];
}

void display(void)
{
    int i;

    for (i = top; i >= 0; i--)
        printf("%d\n", stack[i]);
}

int main(void)
{
    int choice, value;

    stack = (int *)malloc(capacity * sizeof(int));
    if (stack == NULL) {
        printf("Memory not available\n");
        return 1;
    }
    do {
        printf("\n1-PUSH 2-POP 3-DISPLAY 4-EXIT\n");
        printf("Enter your choice: ");
        if (scanf("%d", &choice) != 1)
            break;
        switch (choice) {
        case 1: printf("Enter the value to be pushed: ");
                scanf("%d", &value);
                push(value);
                break;
```

```
            case 2: if (top == -1)
                        printf("Stack Underflow\n");
                    else
                        printf("POPed element is %d\n", pop());
                    break;
            case 3: if (top == -1)
                        printf("Stack is Empty\n");
                    else {
                        printf("Stack status (top to bottom):\n");
                        display();
                    }
                    break;
            }
        } while (choice >= 1 && choice <= 3);

        free(stack);              /* give the memory back */
        return 0;
    }
```

Sample Output:

```
$ ./stack_dynamic
1-PUSH 2-POP 3-DISPLAY 4-EXIT
Enter your choice: 1
Enter the value to be pushed: 10
1-PUSH 2-POP 3-DISPLAY 4-EXIT
Enter your choice: 1
Enter the value to be pushed: 20
1-PUSH 2-POP 3-DISPLAY 4-EXIT
Enter your choice: 1
Enter the value to be pushed: 30
Stack full: size increased to 4
1-PUSH 2-POP 3-DISPLAY 4-EXIT
Enter your choice: 3
Stack status (top to bottom):
30
20
10
1-PUSH 2-POP 3-DISPLAY 4-EXIT
Enter your choice: 2
POPed element is 30
1-PUSH 2-POP 3-DISPLAY 4-EXIT
Enter your choice: 3
Stack status (top to bottom):
20
10
1-PUSH 2-POP 3-DISPLAY 4-EXIT
Enter your choice: 4
```

Representation of Stack using Linked List

A stack can also be implemented using a linked list. Each element is stored in a node having two parts: info (the value) and link (address of the next node). The pointer `top` points to the first node, which is the top of the stack.

* PUSH: create a new node and insert it at the beginning of the list. The new node becomes

the top.

* POP: delete the first node of the list. The next node becomes the top.
* The stack is empty when top = NULL. There is no overflow unless memory itself is exhausted.

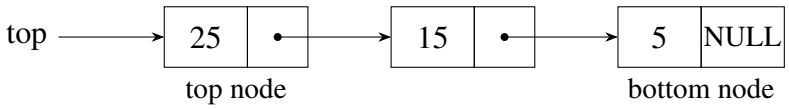


Fig: Stack 25, 15, 5 (25 on top) using a linked list.

Pgm: Write a C program to implement a stack using linked list.

```
/* Stack using a singly linked list */
#include <stdio.h>
#include <stdlib.h>

struct node {
    int info;
    struct node *link;
};

struct node *top = NULL;      /* empty stack */

void push(int value)
{
    struct node *temp;

    temp = (struct node *)malloc(sizeof(struct node));
    if (temp == NULL) {
        printf("Stack Overflow (no memory)\n");
        return;
    }
    temp->info = value;
    temp->link = top;         /* new node points to old top */
    top = temp;               /* new node becomes the top */
}

int pop(void)
{
    struct node *temp = top;
    int x = temp->info;

    top = top->link;          /* next node becomes the top */
    free(temp);               /* release the old top node */
    return x;
}

void display(void)
{
    struct node *p;

    for (p = top; p != NULL; p = p->link)
```

```
        printf("%d -> ", p->info);
    printf("NULL\n");
}

int main(void)
{
    int choice, value;

    do {
        printf("\n1-PUSH 2-POP 3-PEEK 4-DISPLAY 5-EXIT\n");
        printf("Enter your choice: ");
        if (scanf("%d", &choice) != 1)
            break;
        switch (choice) {
        case 1: printf("Enter the value to be pushed: ");
                scanf("%d", &value);
                push(value);
                break;
        case 2: if (top == NULL)
                    printf("Stack Underflow\n");
                else
                    printf("POPed element is %d\n", pop());
                break;
        case 3: if (top == NULL)
                    printf("Stack is Empty\n");
                else
                    printf("Top element is %d\n", top->info);
                break;
        case 4: if (top == NULL)
                    printf("Stack is Empty\n");
                else
                    display();
                break;
        }
    } while (choice >= 1 && choice <= 4);
    return 0;
}
```

Sample Output:

```
$ ./stack_linked
1-PUSH 2-POP 3-PEEK 4-DISPLAY 5-EXIT
Enter your choice: 2
Stack Underflow
1-PUSH 2-POP 3-PEEK 4-DISPLAY 5-EXIT
Enter your choice: 1
Enter the value to be pushed: 5
1-PUSH 2-POP 3-PEEK 4-DISPLAY 5-EXIT
Enter your choice: 1
Enter the value to be pushed: 15
1-PUSH 2-POP 3-PEEK 4-DISPLAY 5-EXIT
Enter your choice: 1
Enter the value to be pushed: 25
1-PUSH 2-POP 3-PEEK 4-DISPLAY 5-EXIT
Enter your choice: 3
Top element is 25
```

```
1-PUSH 2-POP 3-PEEK 4-DISPLAY 5-EXIT
Enter your choice: 4
25 -> 15 -> 5 -> NULL
1-PUSH 2-POP 3-PEEK 4-DISPLAY 5-EXIT
Enter your choice: 2
POPed element is 25
1-PUSH 2-POP 3-PEEK 4-DISPLAY 5-EXIT
Enter your choice: 4
15 -> 5 -> NULL
1-PUSH 2-POP 3-PEEK 4-DISPLAY 5-EXIT
Enter your choice: 5
```

Multiple Stacks using a Single Array

Sometimes a program needs more than one stack. Instead of taking a separate array for each stack, we can keep two stacks in one array so that memory is shared.

* Stack 1 starts at the left end (index 0) and grows towards the right. Initially top1 = −1.
* Stack 2 starts at the right end (index MAX−1) and grows towards the left. Initially top2 = MAX.
* PUSH on stack 1: `arr[++top1] = value;` PUSH on stack 2: `arr[-top2] = value;`
* Overflow occurs only when the two stacks meet, ie. when top1 + 1 = top2. So no space is wasted as long as the array has a free cell.

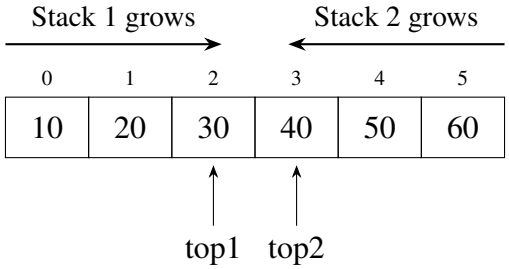


Fig: Two stacks in one array of size 6 (top1 + 1 = top2, so the array is full).

More than two stacks: For n stacks, the array is divided into n equal parts. Each stack i has its own boundary b[i] and top t[i]. Stack i is full when t[i] = b[i+1]. This is simple, but a stack may overflow even when other parts of the array are free.

Pgm: Write a C program to implement two stacks using a single array.

```
/* Two stacks in a single array.
 * Stack 1 grows from the left, stack 2 grows from the right. */
#include <stdio.h>

#define MAX 6

int arr[MAX];
int top1 = -1;              /* stack 1 is empty */
int top2 = MAX;             /* stack 2 is empty */

void push(int s, int value)
```

```
{
    if (top1 + 1 == top2) {          /* no free cell left */
        printf("Stack Overflow\n");
        return;
    }
    if (s == 1)
        arr[++top1] = value;
    else
        arr[--top2] = value;
}

void pop(int s)
{
    if (s == 1) {
        if (top1 == -1)
            printf("Stack 1 Underflow\n");
        else
            printf("POPed from stack 1: %d\n", arr[top1--]);
    } else {
        if (top2 == MAX)
            printf("Stack 2 Underflow\n");
        else
            printf("POPed from stack 2: %d\n", arr[top2++]);
    }
}

void display(void)
{
    int i;

    printf("Stack 1 (top to bottom): ");
    for (i = top1; i >= 0; i--)
        printf("%d ", arr[i]);
    printf("\nStack 2 (top to bottom): ");
    for (i = top2; i < MAX; i++)
        printf("%d ", arr[i]);
    printf("\n");
}

int main(void)
{
    int choice, s, value;

    do {
        printf("\n1-PUSH 2-POP 3-DISPLAY 4-EXIT\n");
        printf("Enter your choice: ");
        if (scanf("%d", &choice) != 1)
            break;
        switch (choice) {
        case 1: printf("Stack number (1/2) and value: ");
                scanf("%d %d", &s, &value);
                push(s, value);
                break;
        case 2: printf("Stack number (1/2): ");
```

```
                scanf("%d", &s);
                pop(s);
                break;
        case 3: display();
                break;
        }
    } while (choice >= 1 && choice <= 3);
    return 0;
}
```

Sample Output:

```
$ ./two_stacks
1-PUSH 2-POP 3-DISPLAY 4-EXIT
Enter your choice: 1
Stack number (1/2) and value: 1 10
1-PUSH 2-POP 3-DISPLAY 4-EXIT
Enter your choice: 1
Stack number (1/2) and value: 1 20
1-PUSH 2-POP 3-DISPLAY 4-EXIT
Enter your choice: 1
Stack number (1/2) and value: 2 60
1-PUSH 2-POP 3-DISPLAY 4-EXIT
Enter your choice: 1
Stack number (1/2) and value: 2 50
1-PUSH 2-POP 3-DISPLAY 4-EXIT
Enter your choice: 1
Stack number (1/2) and value: 2 40
1-PUSH 2-POP 3-DISPLAY 4-EXIT
Enter your choice: 1
Stack number (1/2) and value: 1 30
1-PUSH 2-POP 3-DISPLAY 4-EXIT
Enter your choice: 1
Stack number (1/2) and value: 1 99
Stack Overflow
1-PUSH 2-POP 3-DISPLAY 4-EXIT
Enter your choice: 3
Stack 1 (top to bottom): 30 20 10
Stack 2 (top to bottom): 40 50 60
1-PUSH 2-POP 3-DISPLAY 4-EXIT
Enter your choice: 2
Stack number (1/2): 2
POPed from stack 2: 40
1-PUSH 2-POP 3-DISPLAY 4-EXIT
Enter your choice: 2
Stack number (1/2): 1
POPed from stack 1: 30
1-PUSH 2-POP 3-DISPLAY 4-EXIT
Enter your choice: 3
Stack 1 (top to bottom): 20 10
Stack 2 (top to bottom): 50 60
1-PUSH 2-POP 3-DISPLAY 4-EXIT
Enter your choice: 4
```

Applications of Stack

Because of its LIFO nature, a stack is used whenever the last item saved must be the first one used. The important applications are:

i. Reversing a list
ii. Factorial calculation
iii. Infix to postfix transformation
iv. Evaluating arithmetic (postfix) expressions
v. Recursion (function calls)

Reversing a List

To reverse a list, PUSH all the elements one by one onto a stack, then POP them one by one. The last element pushed comes out first, so the elements come out in reverse order.

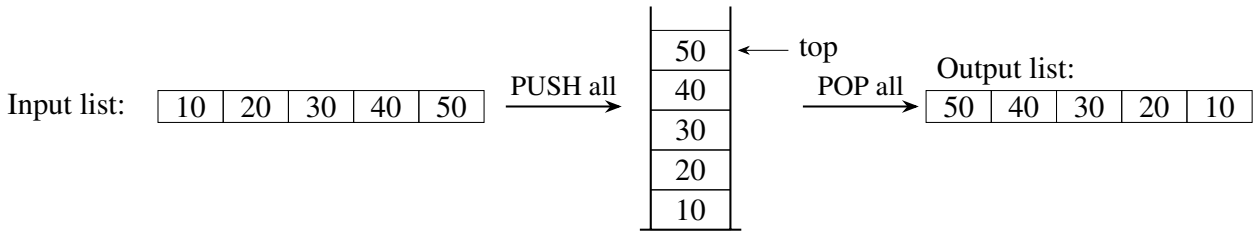


Fig: Reversing a list using a stack.

Pgm: Write a C program to reverse a list of numbers using a stack.

```
/* Reversing a list of numbers using a stack */
#include <stdio.h>

#define MAX 50

int stack[MAX];
int top = -1;

void push(int value)
{
    stack[++top] = value;
}

int pop(void)
{
    return stack[top--];
}

int main(void)
{
    int n, i, value;

    printf("Enter the number of elements: ");
    scanf("%d", &n);
```

```
    printf("Enter %d elements: ", n);
    for (i = 0; i < n; i++) {
        scanf("%d", &value);
        push(value);                    /* push every element */
    }

    printf("Reversed list: ");
    while (top != -1)                   /* pop until empty */
        printf("%d ", pop());
    printf("\n");
    return 0;
}
```

Sample Output:

```
$ ./reverse_list
Enter the number of elements: 5
Enter 5 elements: 10 20 30 40 50
Reversed list: 50 40 30 20 10
```

Factorial Calculation

The factorial of n is $n! = n \times (n-1) \times (n-2) \times \ldots \times 2 \times 1$, and $0! = 1! = 1$.

Using a stack: PUSH n, n−1, . . . , 2 onto the stack. Then POP the elements one by one and multiply them into the result. For 4!: PUSH 4, 3, 2 → POP 2, 3, 4 → result = $1 \times 2 \times 3 \times 4 = 24$.

Using recursion: fact(n) = 1 if n = 0 or 1, otherwise fact(n) = n × fact(n−1). Here the system stack does the same work automatically (see Recursion below).

Pgm: Write a C program to find the factorial of a number using (i) a stack and (ii) recursion.

```
/* Factorial of n: (i) using a stack (ii) using recursion */
#include <stdio.h>

#define MAX 20

int stack[MAX];
int top = -1;

long fact_stack(int n)
{
    long result = 1;

    while (n > 1)                       /* push n, n-1, ..., 2 */
        stack[++top] = n--;
    while (top != -1)                   /* pop and multiply */
        result = result * stack[top--];
    return result;
}

long fact_recursive(int n)
{
    if (n == 0 || n == 1)               /* base case */
        return 1;
    return n * fact_recursive(n - 1); /* recursive case */
```

```
}

int main(void)
{
    int n;

    printf("Enter a number: ");
    scanf("%d", &n);
    if (n < 0 || n >= MAX) {
        printf("Enter a number from 0 to %d\n", MAX - 1);
        return 1;
    }
    printf("Factorial using stack     = %ld\n", fact_stack(n));
    printf("Factorial using recursion = %ld\n", fact_recursive(n));
    return 0;
}
```

Sample Output:

```
$ ./factorial
Enter a number: 5
Factorial using stack     = 120
Factorial using recursion = 120
```

Infix to Postfix Transformation

An arithmetic expression can be written in three forms:

Form	Operator position	Example
Infix	between the operands	A + B
Prefix (Polish)	before the operands	+ A B
Postfix (Reverse Polish)	after the operands	A B +

Humans write infix expressions, but a computer evaluates postfix expressions more easily, because a postfix expression needs no brackets and no precedence rules. So the compiler first converts infix to postfix using a stack.

Operator Precedence Table

Each symbol gets two precedence values: F (stack precedence, used when the symbol is on top of the stack) and G (input precedence, used when the symbol is the incoming symbol).

Symbol	F (Stack precedence)	G (Input precedence)	Associativity
+ , −	2	1	Left to right
∗ , / , %	4	3	Left to right
ˆ or $	5	6	Right to left
Operand (A, B, . . .)	8	7	–
(	0	9	–
)	–	0	–
#	−1	–	–

Rules: when to PUSH and when to POP

Compare **F(stack top)** with **G(incoming symbol)**:

Condition	Action
F(top) > G(symbol)	**POP** the stack top to the postfix. Repeat with the new top.
F(top) < G(symbol)	**PUSH** the incoming symbol.
F(top) = G(symbol)	**POP and discard**. Happens only when '(' meets ')'.
End of input	**POP** all remaining symbols until '#'.

Memory aid: Stack top stronger: POP. Incoming stronger: PUSH. Equal: discard the brackets.

Algorithm

1. Push '#' onto the stack (bottom marker).
2. Scan the infix expression from left to right. For each symbol:
   (a) while F(stack top) > G(symbol), pop the top and add it to postfix;
   (b) if F(stack top) is not equal to G(symbol), push the symbol, else pop and discard '('.
3. After the scan, pop and add all symbols to postfix until '#' is reached.

Example: A + B ∗ C − (D / E ˆ F) ∗ G

Step	Symbol	Action [F(top) vs G(symbol)]	Stack	Postfix
0	–	Push #	#	
1	A	F(#)=−1 < 7 : push A	# A	
2	+	F(A)=8 > 1 : pop A; F(#)=−1 < 1 : push +	# +	A
3	B	F(+)=2 < 7 : push B	# + B	A
4	∗	F(B)=8 > 3 : pop B; F(+)=2 < 3 : push ∗	# + ∗	AB
5	C	F(∗)=4 < 7 : push C	# + ∗ C	AB
6	−	pop C (8>1), pop ∗ (4>1), pop + (2>1); F(#)=−1 < 1 : push −	# −	ABC∗+
7	(	F(−)=2 < 9 : push (	# − (	ABC∗+

Step	Symbol	Action [F(top) vs G(symbol)]	Stack	Postfix
8	D	F(()=0 < 7 : push D	# – (D	ABC∗+
9	/	F(D)=8 > 3 : pop D; F(()=0 < 3 : push /	# – (/	ABC∗+D
10	E	F(/)=4 < 7 : push E	# – (/ E	ABC∗+D
11	^	F(E)=8 > 6 : pop E; F(/)=4 < 6 : push ^	# – (/ ^	ABC∗+DE
12	F	F(^)=5 < 7 : push F	# – (/ ^ F	ABC∗+DE
13	)	pop F, ^, / (all > 0); F(()=0 = G())=0 : discard (	# –	ABC∗+DEF^/
14	∗	F(−)=2 < 3 : push ∗	# – ∗	ABC∗+DEF^/
15	G	F(∗)=4 < 7 : push G	# – ∗ G	ABC∗+DEF^/
16	End	pop G, ∗, – until #	#	**ABC∗+DEF^/G∗–**

Postfix: A B C ∗ + D E F ^ / G ∗ –

Pgm: Write a C program to convert an infix expression to postfix using the operator precedence table.

```
/*
 * Infix to Postfix using an operator precedence table
 * F(x) = stack precedence (symbol already on the stack)
 * G(x) = input precedence (incoming symbol)
 */
#include <stdio.h>
#include <string.h>

#define MAX 100

char stack[MAX];
int top = -1;

void push(char c)
{
    if (top == MAX - 1) {
        printf("Stack overflow\n");
        return;
    }
    stack[++top] = c;
}

char pop(void)
{
    return stack[top--];
}

/* Stack precedence */
int F(char symbol)
{
    switch (symbol) {
    case '+': case '-':            return 2;
    case '*': case '/': case '%':  return 4;
    case '^': case '$':            return 5;
    case '(':                      return 0;
    case '#':                      return -1;
```

```
    default:                        return 8;   /* operand */
    }
}

/* Input precedence */
int G(char symbol)
{
    switch (symbol) {
    case '+': case '-':             return 1;
    case '*': case '/': case '%':   return 3;
    case '^': case '$':             return 6;
    case '(':                       return 9;
    case ')':                       return 0;
    default:                        return 7;   /* operand */
    }
}

void infix_to_postfix(const char infix[], char postfix[])
{
    int i, j = 0;
    char symbol;

    top = -1;
    push('#');                  /* bottom-of-stack marker */

    for (i = 0; infix[i] != '\0'; i++) {
        symbol = infix[i];
        if (symbol == ' ')
            continue;

        /* Pop while stack top outranks the input */
        while (F(stack[top]) > G(symbol))
            postfix[j++] = pop();

        if (F(stack[top]) != G(symbol))
            push(symbol);       /* push input symbol */
        else
            pop();              /* '(' meets ')' : discard */
    }

    /* Pop remaining symbols */
    while (stack[top] != '#')
        postfix[j++] = pop();

    postfix[j] = '\0';
}

int main(void)
{
    char infix[MAX], postfix[MAX];

    printf("Enter a valid infix expression: ");
    scanf("%99s", infix);
```

```
    infix_to_postfix(infix, postfix);

    printf("Postfix expression: %s\n", postfix);
    return 0;
}
```

Sample Output:

```
$ ./infix_to_postfix
Enter a valid infix expression: A+B*C-(D/E^F)*G
Postfix expression: ABC*+DEF^/G*-
```

Evaluating Arithmetic Expressions

A postfix expression is evaluated using a stack of operands.

Algorithm

Scan the postfix expression from left to right. For each symbol:

- If it is an operand, PUSH it onto the stack.
- If it is an operator, POP the top element as **op2** and the next element as **op1**. Compute **op1 operator op2** and PUSH the result.

At the end, the only value left on the stack is the result. Note that the first value popped is the right operand; this matters for –, /, % and ^.

Example: 6 2 3 + – 3 8 2 / + ∗ 2 ^ 3 +

Step	Symbol	op1	op2	Result	Stack (bottom → top)
1	6				6
2	2				6, 2
3	3				6, 2, 3
4	+	2	3	2 + 3 = 5	6, 5
5	–	6	5	6 – 5 = 1	1
6	3				1, 3
7	8				1, 3, 8
8	2				1, 3, 8, 2
9	/	8	2	8 / 2 = 4	1, 3, 4
10	+	3	4	3 + 4 = 7	1, 7
11	∗	1	7	1 ∗ 7 = 7	7
12	2				7, 2
13	^	7	2	7 ^ 2 = 49	49
14	3				49, 3
15	+	49	3	49 + 3 = 52	52

Result = 52

Pgm: Write a C program to evaluate a postfix expression (single-digit operands) using a stack.

```
/* Evaluation of a postfix expression using a stack */
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>

#define MAX 50

int stack[MAX];
int top = -1;

void push(int item)
{
    if (top == MAX - 1) {
        printf("Stack overflow\n");
        exit(1);
    }
    stack[++top] = item;
}

int pop(void)
{
    if (top == -1) {
        printf("Invalid postfix expression\n");
        exit(1);
    }
    return stack[top--];
}

/* Returns base raised to exp (exp >= 0) */
int power(int base, int exp)
{
    int result = 1;

    while (exp-- > 0)
        result = result * base;
    return result;
}

/* Applies operator op to op1 and op2 */
int compute(char op, int op1, int op2)
{
    if ((op == '/' || op == '%') && op2 == 0) {
        printf("Division by zero\n");
        exit(1);
    }
    switch (op) {
    case '+': return op1 + op2;
    case '-': return op1 - op2;
    case '*': return op1 * op2;
    case '/': return op1 / op2;
    case '%': return op1 % op2;
    case '^': return power(op1, op2);
    default:
        printf("Invalid operator %c\n", op);
```

```
            exit(1);
        }
    }

    int main(void)
    {
        char postfix[MAX], symbol;
        int i, op1, op2, result;

        printf("Enter a valid postfix expression: ");
        scanf("%49s", postfix);

        for (i = 0; postfix[i] != '\0'; i++) {
            symbol = postfix[i];
            if (isdigit(symbol)) {
                push(symbol - '0');          /* operand */
            } else {
                op2 = pop();                 /* right operand */
                op1 = pop();                 /* left operand */
                result = compute(symbol, op1, op2);
                printf("%d %c %d = %d\n", op1, symbol, op2, result);
                push(result);
            }
        }

        result = pop();
        if (top != -1) {
            printf("Invalid postfix expression\n");
            return 1;
        }
        printf("Result = %d\n", result);
        return 0;
    }
```

Sample Output:

```
$ ./postfix_evaluation
Enter a valid postfix expression: 623+-382/+*2^3+
2 + 3 = 5
6 - 5 = 1
8 / 2 = 4
3 + 4 = 7
1 * 7 = 7
7 ^ 2 = 49
49 + 3 = 52
Result = 52
```

Recursion

A recursive function is a function that calls itself. Every recursive function has two parts:

* Base case: a condition where the answer is known directly, so the function stops calling itself.
* Recursive case: the function calls itself with a smaller input, moving closer to the base case.

How recursion uses a stack

Every time a function is called, the system pushes an activation record (its parameters, local variables and return address) onto the system stack. When the function returns, its record is popped. So the call made last returns first (LIFO).

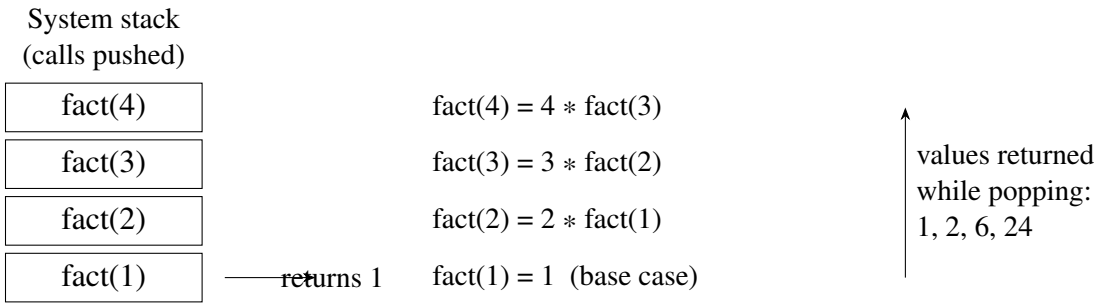


Fig: Calls of fact(4) on the system stack.

Tower of Hanoi

There are three pegs A, B and C. n disks of different sizes are placed on peg A, largest at the bottom. The task is to move all disks to peg C using peg B, following these rules:

i. Only one disk can be moved at a time.

ii. Only the top disk of a peg can be moved.

iii. A larger disk can never be placed on a smaller disk.

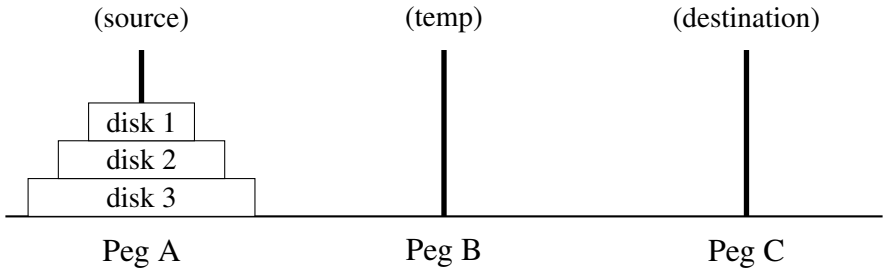


Fig: Initial state of Tower of Hanoi with 3 disks.

Recursive solution to move n disks from source to destination using temp:

1. Move the top n−1 disks from source to temp (using destination).
2. Move disk n (the largest) from source to destination.
3. Move the n−1 disks from temp to destination (using source).

The base case is n = 0 (nothing to move). The total number of moves is $\boxed{2^n - 1}$. For 3 disks it is $2^3 - 1 = 7$ moves.

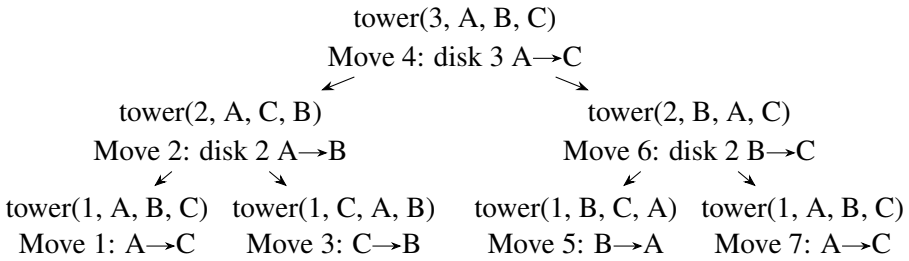


Fig: Recursion tree for 3 disks (tower(n, source, temp, dest)).

Pgm: Write a C program to solve the Tower of Hanoi problem using recursion.

```
/* Tower of Hanoi using recursion */
#include <stdio.h>

int count = 0;                  /* total number of moves */

/* Move n disks from source to dest using temp */
void tower(int n, char source, char temp, char dest)
{
    if (n == 0)                 /* base case: nothing to move */
        return;

    tower(n - 1, source, dest, temp);   /* n-1 disks to temp */

    count++;
    printf("Move %2d: disk %d from %c to %c\n",
           count, n, source, dest);

    tower(n - 1, temp, source, dest);   /* n-1 disks to dest */
}

int main(void)
{
    int n;

    printf("Enter the number of disks: ");
    scanf("%d", &n);

    printf("Sequence of moves (A to C using B):\n");
    tower(n, 'A', 'B', 'C');

    printf("Total number of moves = %d\n", count);
    return 0;
}
```

Sample Output:

```
$ ./tower_of_hanoi
Enter the number of disks: 3
Sequence of moves (A to C using B):
Move  1: disk 1 from A to C
Move  2: disk 2 from A to B
Move  3: disk 1 from C to B
Move  4: disk 3 from A to C
Move  5: disk 1 from B to A
```

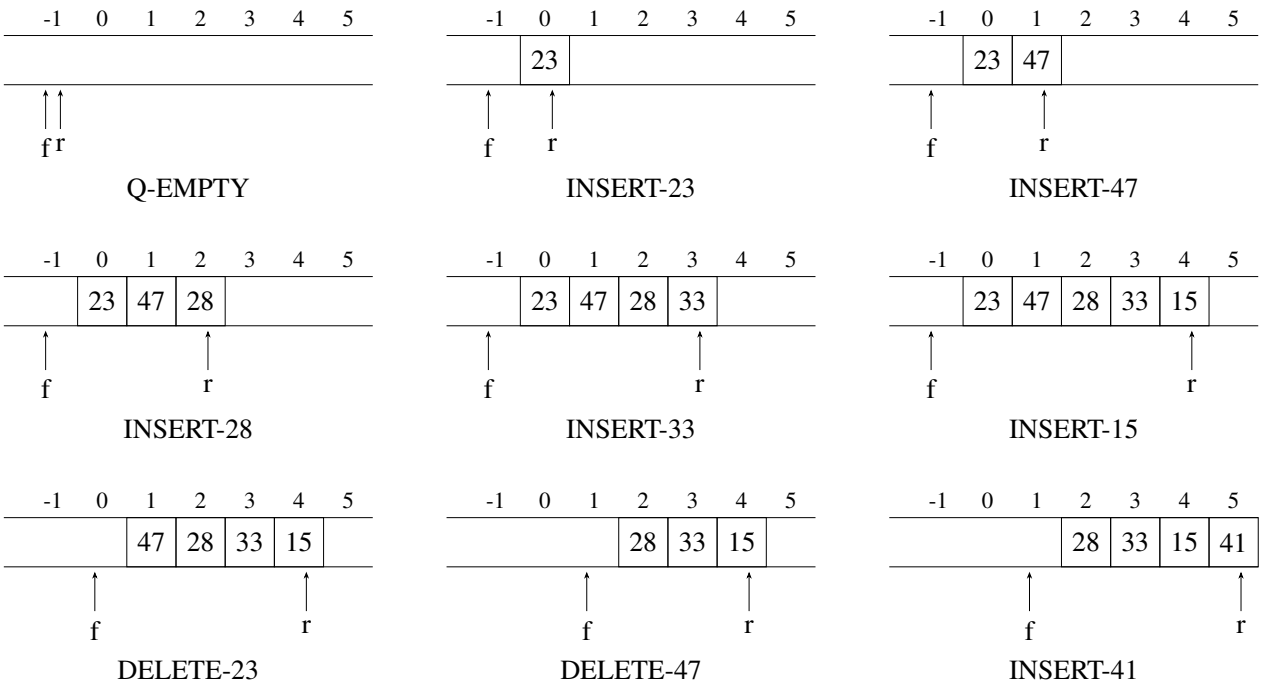
```
Move  6: disk 2 from B to C
Move  7: disk 1 from A to C
Total number of moves = 7
```

Queue:

A queue is a non-primitive linear data structure, which is an ordered list (array) in which insertions and deletions take place at different ends. The end at which new elements are added/inserted is called rear and that from which old/existing elements are deleted is called the front.

The restrictions on a queue imply that if we insert 23, 47, 28, 33 and 15 in that order, then 23 is the first element deleted from the queue. Since the first element inserted into a queue is the first element deleted, queue are also known as First-In-First-Out (FIFO) array/list.

Below figure illustrate different operation on queue:



where f: Queue Front r: Queue Rear

Fig: Insertion and Deletion elements in a queue.

Array representation of Queue.

Queue can be easily represented using linear array. As stated earlier, every queue has FRONT and REAR variables that point to the position from where deletions and insertions can be done, respectively. The REAR is the position where the elements will be inserted and FRONT is the position, where the elements will be deleted. There is another variable MAX, which is used to store the maximum number of elements that the queue can hold.

Initial value of front and rear is −1
ie. if front = rear : it indicates queue is empty
if rear = MAX−1 : it indicates queue is full. it has reached last location, no more

insertion possible
(MAX−1, it is because array index start from 0).

The easiest way to implement Queue using 1-D array

```
#define MAX 5
int front = -1;
int rear = -1;
int queue[MAX];
```

Note: Declare front & rear as global variable. Because these variables are used in many queue functions.

where MAX : Maximum no of entries.
queue[0] : First element of queue
queue[1] : Next (or) second element of queue.
queue[MAX−1] : Last element of stack.
front = −1, rear = −1 | To denote an empty queue.

ie. initial value of front & rear is −1.

Error Condition on Queue

i. Queue Overflow → if rear = MAX−1
ii. Queue Underflow → if front = rear
iii. Queue Empty → if front = rear.

Queue Insert Operations:

The INSERT operation is used to insert an element into the queue. The new element is inserted at the rear end of the queue. ie. REAR pointer will be incremented by 1 and the value will be stored at the position pointed by rear. However, before inserting an element in a queue, we must check for OVERFLOW condition. An overflow will occur when we try to insert an element into a queue that is already full. Where rear = MAX−1 we have an overflow condition.

Overflow Function.

```
int queue_overflow()
{ if (rear == MAX-1)
     return 1;
  else
     return 0;
} /* End of overflow */
```

where rear is a global variable. so no need to pass as a function parameter

Queue Insert Function.

```
void queue_insert(int queue[], int
   value)
{ rear = rear+1;
  queue[rear] = value;
  return;
} /* End of Insert */
```

(or)
```
queue[++rear] = value;
  return;
```

Delete (queue) Function:

The DELETE operation is used to delete a front element from the queue. To delete an element from the queue, the value of FRONT will be incremented by 1 and the value at the position pointed by front will be deleted. However, before deleting an element from a queue, we must check for queue underflow condition. An underflow will occur when we try to delete an element from a queue that is already empty. Where front = rear we have an underflow condition.

queue Underflow Function.

```
int queue_underflow()
{
  if (front == rear)
     return 1;
  else
     return 0;
} /* End of underflow */
```

where front and rear var are declared as global variables

Queue Delete Function:

```
int queue_delete(int queue[])
{ int x;
  front = front+1;
  x = queue[front];
  return (x);
}
```

(or) `return(queue[++front]);`

Queue Display Function.

Display function displays the status of the queue from FRONT element to REAR element. Display function also checks for queue empty using FRONT = REAR, print appropriate message "Queue is Empty" else returns the content of queue.

Queue Empty Function.

```
int queue_empty()
{
  if (front == rear)
     return 1;
  else
     return 0;
} /* End of empty */
```

Queue Display Function

```
void queue_display(int queue[])
{ int i;
  for(i=front+1; i<=rear; i++)
     printf("%d \t", queue[i]);
  printf("\n");
  return;
} /* End of queue display */
```

Pgm: Write a 'C' Program to perform INSERT, DELETE and DISPLAY operations on queue using static array.

```
#include<stdio.h>
#include<stdlib.h>
#define MAX 5
int front =-1;
int rear =-1;
int main()
{ int queue_overflow();
  int queue_underflow();
  int queue_empty();
  void queue_insert(int queue[], int value);
  int queue_delete(int queue[]);
  void queue_display(int queue[]);
  int queue[MAX], choice, x, value;
  do
  { printf("Enter 1-INSERT \n 2-DELETE \n
           3-DISPLAY \n Any value to EXIT \n");
    scanf("%d", &choice);
    switch(case)
    { case 1: if (queue_overflow())
                 printf("Queue Overflow \n");
              else
              { printf("Enter the value to be
                       inserted \n");
                scanf("%d", &value);
                queue_insert(queue, value);
              }
              break;
      case 2: if (queue_underflow())
                 printf("Queue Underflow \n");
              else
              { x = queue_delete(queue);
                printf("Deleted element from Queue
                       is %d \n", x);
              }
              break;
      case 3: if (queue_empty())
                 printf("Queue is Empty \n");
              else
              { printf("Queue status \n");
                queue_display(queue);
              }
```

```
            break;
      default: printf("Invalid input - EXIT \n");
               exit(0);
    } /* End of switch-case */
  } while (choice >=1 && choice <=3);
    /* End of do-while */

  return 0;
} /* End of main program */
```

Representation of Queue using Linked List

A queue can also be implemented using a linked list. Two pointers are kept: front points to the first node (deletions happen here) and rear points to the last node (insertions happen here).

* INSERT: create a new node and attach it after the rear node. The new node becomes the rear.
* DELETE: remove the front node. The next node becomes the front.
* The queue is empty when front = NULL. There is no fixed size, so no overflow (until memory runs out).

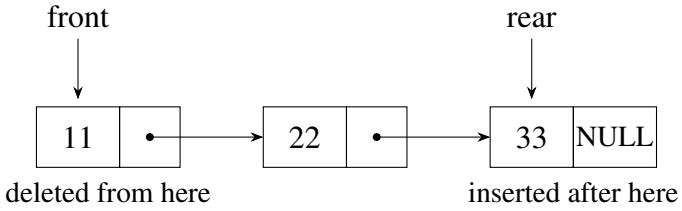


Fig: Queue 11, 22, 33 using a linked list.

Pgm: Write a C program to implement a queue using linked list.

```
/* Queue using a singly linked list */
#include <stdio.h>
#include <stdlib.h>

struct node {
    int info;
    struct node *link;
};

struct node *front = NULL, *rear = NULL;     /* empty queue */

void insert(int value)
{
    struct node *temp;

    temp = (struct node *)malloc(sizeof(struct node));
    if (temp == NULL) {
        printf("Queue Overflow (no memory)\n");
        return;
    }
    temp->info = value;
```

```
    temp->link = NULL;
    if (rear == NULL)             /* first node */
        front = rear = temp;
    else {
        rear->link = temp;        /* attach at the rear */
        rear = temp;
    }
}

int delete(void)
{
    struct node *temp = front;
    int x = temp->info;

    front = front->link;          /* next node becomes front */
    if (front == NULL)            /* queue became empty */
        rear = NULL;
    free(temp);
    return x;
}

void display(void)
{
    struct node *p;

    for (p = front; p != NULL; p = p->link)
        printf("%d ", p->info);
    printf("\n");
}

int main(void)
{
    int choice, value;

    do {
        printf("\n1-INSERT 2-DELETE 3-DISPLAY 4-EXIT\n");
        printf("Enter your choice: ");
        if (scanf("%d", &choice) != 1)
            break;
        switch (choice) {
        case 1: printf("Enter the value to be inserted: ");
                scanf("%d", &value);
                insert(value);
                break;
        case 2: if (front == NULL)
                    printf("Queue Underflow\n");
                else
                    printf("Deleted element is %d\n", delete());
                break;
        case 3: if (front == NULL)
                    printf("Queue is Empty\n");
                else {
                    printf("Queue (front to rear): ");
                    display();
```

```
                }
                break;
            }
        } while (choice >= 1 && choice <= 3);
        return 0;
    }
```

Sample Output:

```
$ ./queue_linked
1-INSERT 2-DELETE 3-DISPLAY 4-EXIT
Enter your choice: 2
Queue Underflow
1-INSERT 2-DELETE 3-DISPLAY 4-EXIT
Enter your choice: 1
Enter the value to be inserted: 11
1-INSERT 2-DELETE 3-DISPLAY 4-EXIT
Enter your choice: 1
Enter the value to be inserted: 22
1-INSERT 2-DELETE 3-DISPLAY 4-EXIT
Enter your choice: 1
Enter the value to be inserted: 33
1-INSERT 2-DELETE 3-DISPLAY 4-EXIT
Enter your choice: 3
Queue (front to rear): 11 22 33
1-INSERT 2-DELETE 3-DISPLAY 4-EXIT
Enter your choice: 2
Deleted element is 11
1-INSERT 2-DELETE 3-DISPLAY 4-EXIT
Enter your choice: 3
Queue (front to rear): 22 33
1-INSERT 2-DELETE 3-DISPLAY 4-EXIT
Enter your choice: 4
```

Circular Queue

Limitation of the ordinary (linear) queue

In the array queue, rear only moves forward. Once rear = MAX−1, the queue reports overflow, even if some elements were deleted from the front and those cells are now free. The free cells at the front can never be used again.

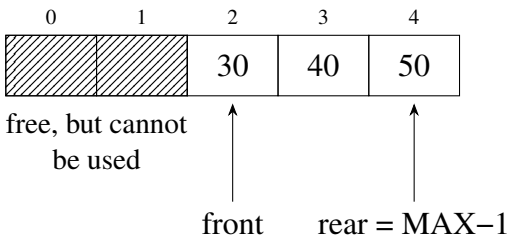


Fig: Linear queue reports “full” although two cells are free.

Circular Queue

In a circular queue the last position is connected back to the first position, so the array is treated as a circle. After index MAX−1 comes index 0 again. This is done using the modulus (%) operator:

rear = (rear + 1) % MAX	front = (front + 1) % MAX

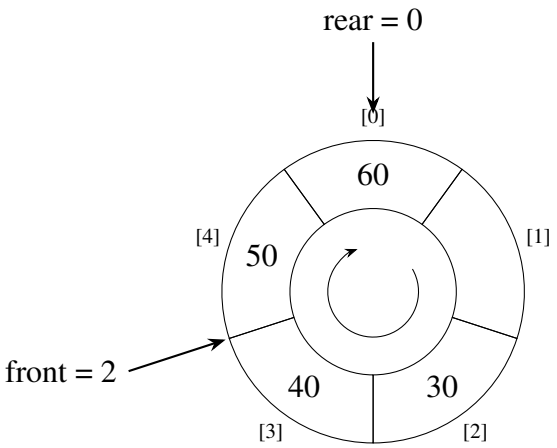


Fig: Circular queue of size 5. After 50 at [4], the next insert 60 goes to [0].

Conditions (front = rear = −1 initially)

Condition	**Test**
Queue empty / underflow	front = −1
Queue full / overflow	(rear + 1) % MAX = front
Only one element	front = rear

Algorithm: Insert

1. If (rear + 1) % MAX = front, print "Queue overflow" and stop.
2. If front = −1 (queue was empty), set front = 0.
3. rear = (rear + 1) % MAX; q[rear] = item.

Algorithm: Delete

1. If front = −1, print "Queue underflow" and stop.
2. item = q[front].
3. If front = rear (last element), set front = rear = −1; else front = (front + 1) % MAX.

Pgm: Write a C program to implement a circular queue using an array.

```
/* Circular queue of integers using an array */
#include <stdio.h>

#define MAX 5

int q[MAX];
```

```
int front = -1, rear = -1;

void insert(int item)
{
    if ((rear + 1) % MAX == front) {
        printf("Queue overflow\n");
        return;
    }
    if (front == -1)                /* queue was empty */
        front = 0;
    rear = (rear + 1) % MAX;
    q[rear] = item;
    printf("%d inserted at position %d\n", item, rear);
}

void delete(void)
{
    if (front == -1) {
        printf("Queue underflow\n");
        return;
    }
    printf("%d deleted from position %d\n", q[front], front);
    if (front == rear)              /* last element removed */
        front = rear = -1;
    else
        front = (front + 1) % MAX;
}

void display(void)
{
    int i;

    if (front == -1) {
        printf("Queue is empty\n");
        return;
    }
    printf("Queue (front to rear): ");
    for (i = front; ; i = (i + 1) % MAX) {
        printf("%d ", q[i]);
        if (i == rear)
            break;
    }
    printf("\nfront = %d, rear = %d\n", front, rear);
}

int main(void)
{
    int choice, item;

    for (;;) {
        printf("\n1.Insert 2.Delete 3.Display 4.Exit\n");
        printf("Enter your choice: ");
        if (scanf("%d", &choice) != 1)
            return 0;
```

```
            switch (choice) {
            case 1:
                printf("Enter the item: ");
                scanf("%d", &item);
                insert(item);
                break;
            case 2:
                delete();
                break;
            case 3:
                display();
                break;
            case 4:
                return 0;
            default:
                printf("Invalid choice\n");
            }
        }
    }
```

Sample Output:

```
$ ./circular_queue
1.Insert 2.Delete 3.Display 4.Exit
Enter your choice: 1
Enter the item: 10
10 inserted at position 0
1.Insert 2.Delete 3.Display 4.Exit
Enter your choice: 1
Enter the item: 20
20 inserted at position 1
1.Insert 2.Delete 3.Display 4.Exit
Enter your choice: 1
Enter the item: 30
30 inserted at position 2
1.Insert 2.Delete 3.Display 4.Exit
Enter your choice: 1
Enter the item: 40
40 inserted at position 3
1.Insert 2.Delete 3.Display 4.Exit
Enter your choice: 1
Enter the item: 50
50 inserted at position 4
1.Insert 2.Delete 3.Display 4.Exit
Enter your choice: 1
Enter the item: 60
Queue overflow
1.Insert 2.Delete 3.Display 4.Exit
Enter your choice: 2
10 deleted from position 0
1.Insert 2.Delete 3.Display 4.Exit
Enter your choice: 2
20 deleted from position 1
1.Insert 2.Delete 3.Display 4.Exit
Enter your choice: 1
Enter the item: 60
60 inserted at position 0
1.Insert 2.Delete 3.Display 4.Exit
```

```
Enter your choice: 3
Queue (front to rear): 30 40 50 60
front = 2, rear = 0
1.Insert 2.Delete 3.Display 4.Exit
Enter your choice: 4
```

Double Ended Queue (Deque)

A deque (pronounced "deck") is a linear list in which insertions and deletions can be done at both ends, front and rear. It is usually implemented as a circular array.

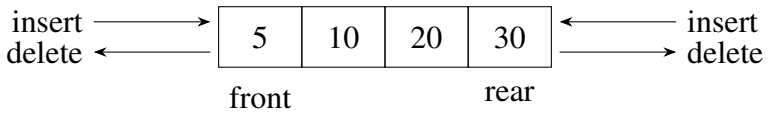


Fig: Deque: insert and delete at both ends.

Types of deque:

- Input-restricted deque: insertion only at one end (rear), deletion at both ends.
- Output-restricted deque: deletion only at one end (front), insertion at both ends.

Operations: insert_front, insert_rear, delete_front, delete_rear and display. With front and rear on a circular array:

- insert_front: front = (front − 1 + MAX) % MAX insert_rear: rear = (rear + 1) % MAX
- delete_front: front = (front + 1) % MAX delete_rear: rear = (rear − 1 + MAX) % MAX
- Full when (rear + 1) % MAX = front; empty when front = −1.

Pgm: Write a C program to implement a double ended queue.

```
/* Double ended queue (deque) using a circular array */
#include <stdio.h>

#define MAX 5

int dq[MAX];
int front = -1, rear = -1;

int is_full(void)
{
    return (rear + 1) % MAX == front;
}

int is_empty(void)
{
    return front == -1;
}

void insert_front(int item)
{
```

```
    if (is_full()) {
        printf("Deque overflow\n");
        return;
    }
    if (is_empty())
        front = rear = 0;
    else
        front = (front - 1 + MAX) % MAX;
    dq[front] = item;
    printf("%d inserted at front\n", item);
}

void insert_rear(int item)
{
    if (is_full()) {
        printf("Deque overflow\n");
        return;
    }
    if (is_empty())
        front = rear = 0;
    else
        rear = (rear + 1) % MAX;
    dq[rear] = item;
    printf("%d inserted at rear\n", item);
}

void delete_front(void)
{
    if (is_empty()) {
        printf("Deque underflow\n");
        return;
    }
    printf("%d deleted from front\n", dq[front]);
    if (front == rear)
        front = rear = -1;
    else
        front = (front + 1) % MAX;
}

void delete_rear(void)
{
    if (is_empty()) {
        printf("Deque underflow\n");
        return;
    }
    printf("%d deleted from rear\n", dq[rear]);
    if (front == rear)
        front = rear = -1;
    else
        rear = (rear - 1 + MAX) % MAX;
}

void display(void)
{
```

```
    int i;

    if (is_empty()) {
        printf("Deque is empty\n");
        return;
    }
    printf("Deque (front to rear): ");
    for (i = front; ; i = (i + 1) % MAX) {
        printf("%d ", dq[i]);
        if (i == rear)
            break;
    }
    printf("\n");
}

int main(void)
{
    int choice, item;

    for (;;) {
        printf("\n1.InsertFront 2.InsertRear 3.DeleteFront\n");
        printf("4.DeleteRear 5.Display 6.Exit\n");
        printf("Enter your choice: ");
        if (scanf("%d", &choice) != 1)
            return 0;
        switch (choice) {
        case 1:
        case 2:
            printf("Enter the item: ");
            scanf("%d", &item);
            if (choice == 1)
                insert_front(item);
            else
                insert_rear(item);
            break;
        case 3:
            delete_front();
            break;
        case 4:
            delete_rear();
            break;
        case 5:
            display();
            break;
        case 6:
            return 0;
        default:
            printf("Invalid choice\n");
        }
    }
}
```

Sample Output:

```
$ ./deque
```

```
1.InsertFront 2.InsertRear 3.DeleteFront
4.DeleteRear 5.Display 6.Exit
Enter your choice: 2
Enter the item: 10
10 inserted at rear
1.InsertFront 2.InsertRear 3.DeleteFront
4.DeleteRear 5.Display 6.Exit
Enter your choice: 2
Enter the item: 20
20 inserted at rear
1.InsertFront 2.InsertRear 3.DeleteFront
4.DeleteRear 5.Display 6.Exit
Enter your choice: 1
Enter the item: 5
5 inserted at front
1.InsertFront 2.InsertRear 3.DeleteFront
4.DeleteRear 5.Display 6.Exit
Enter your choice: 5
Deque (front to rear): 5 10 20
1.InsertFront 2.InsertRear 3.DeleteFront
4.DeleteRear 5.Display 6.Exit
Enter your choice: 3
5 deleted from front
1.InsertFront 2.InsertRear 3.DeleteFront
4.DeleteRear 5.Display 6.Exit
Enter your choice: 4
20 deleted from rear
1.InsertFront 2.InsertRear 3.DeleteFront
4.DeleteRear 5.Display 6.Exit
Enter your choice: 5
Deque (front to rear): 10
1.InsertFront 2.InsertRear 3.DeleteFront
4.DeleteRear 5.Display 6.Exit
Enter your choice: 6
```

Priority Queue

A priority queue is a queue in which every element has a priority. The element with the highest priority is deleted first. Elements with the same priority are served in the order they arrived (FIFO).

Types:

* Ascending priority queue: the element with the smallest priority number is deleted first.
* Descending priority queue: the element with the largest priority number is deleted first.

Example: In a hospital, a patient in critical condition is treated before others, even if they came later. An operating system runs high-priority processes first.

Array implementation: keep the array sorted by priority. INSERT shifts lower-priority elements one place right and places the new element in its correct position. DELETE always removes the element at index 0.

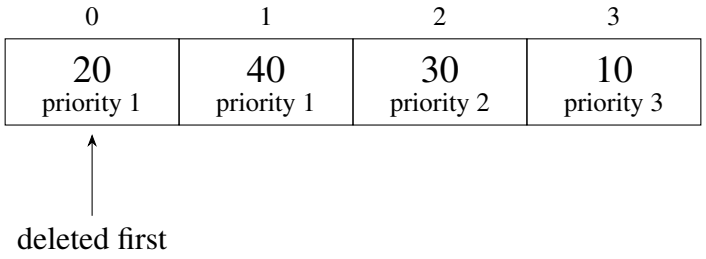


Fig: Ascending priority queue after inserting 10(3), 20(1), 30(2), 40(1).

Pgm: Write a C program to implement a priority queue using an array.

```
/* Priority queue using an ordered array.
 * A smaller priority number means a higher priority.
 * Elements with equal priority are served in FIFO order. */
#include <stdio.h>

#define MAX 5

struct element {
    int item;
    int priority;
};

struct element pq[MAX];
int count = 0;                      /* number of elements */

void insert(int item, int priority)
{
    int i;

    if (count == MAX) {
        printf("Priority queue overflow\n");
        return;
    }
    /* shift lower-priority elements one place right */
    i = count - 1;
    while (i >= 0 && pq[i].priority > priority) {
        pq[i + 1] = pq[i];
        i--;
    }
    pq[i + 1].item = item;
    pq[i + 1].priority = priority;
    count++;
    printf("%d inserted with priority %d\n", item, priority);
}

void delete(void)
{
    int i;

    if (count == 0) {
        printf("Priority queue underflow\n");
        return;
    }
```

```
    printf("%d deleted (priority %d)\n",
           pq[0].item, pq[0].priority);
    for (i = 1; i < count; i++)     /* close the gap */
        pq[i - 1] = pq[i];
    count--;
}

void display(void)
{
    int i;

    if (count == 0) {
        printf("Priority queue is empty\n");
        return;
    }
    printf("Item(Priority): ");
    for (i = 0; i < count; i++)
        printf("%d(%d) ", pq[i].item, pq[i].priority);
    printf("\n");
}

int main(void)
{
    int choice, item, priority;

    for (;;) {
        printf("\n1.Insert 2.Delete 3.Display 4.Exit\n");
        printf("Enter your choice: ");
        if (scanf("%d", &choice) != 1)
            return 0;
        switch (choice) {
        case 1:
            printf("Enter item and priority: ");
            scanf("%d %d", &item, &priority);
            insert(item, priority);
            break;
        case 2:
            delete();
            break;
        case 3:
            display();
            break;
        case 4:
            return 0;
        default:
            printf("Invalid choice\n");
        }
    }
}
```

Sample Output:

```
$ ./priority_queue
1.Insert 2.Delete 3.Display 4.Exit
Enter your choice: 1
```

```
Enter item and priority: 10 3
10 inserted with priority 3
1.Insert 2.Delete 3.Display 4.Exit
Enter your choice: 1
Enter item and priority: 20 1
20 inserted with priority 1
1.Insert 2.Delete 3.Display 4.Exit
Enter your choice: 1
Enter item and priority: 30 2
30 inserted with priority 2
1.Insert 2.Delete 3.Display 4.Exit
Enter your choice: 1
Enter item and priority: 40 1
40 inserted with priority 1
1.Insert 2.Delete 3.Display 4.Exit
Enter your choice: 3
Item(Priority): 20(1) 40(1) 30(2) 10(3)
1.Insert 2.Delete 3.Display 4.Exit
Enter your choice: 2
20 deleted (priority 1)
1.Insert 2.Delete 3.Display 4.Exit
Enter your choice: 3
Item(Priority): 40(1) 30(2) 10(3)
1.Insert 2.Delete 3.Display 4.Exit
Enter your choice: 4
```

Applications of Queues

* CPU scheduling in operating systems (Round Robin).
* Printer queue: documents are printed in the order they are sent.
* Keyboard buffer: keys are processed in the order they are pressed.
* Customer service and ticket counters, call centres.
* Breadth First Search (BFS) in graphs.

Round Robin Algorithm

Round Robin (RR) is a CPU scheduling algorithm used in time-sharing systems. Every process gets the CPU for a fixed small time called the time quantum (time slice). The ready processes are kept in a circular queue.

1. All processes are placed in the ready queue.
2. The process at the front is deleted and runs for one time quantum (or less, if it needs less).
3. If the process is not finished, it is inserted again at the rear of the queue.
4. If it is finished, its completion time is noted.
5. Repeat until the queue is empty.

Turnaround time = completion time − arrival time; Waiting time = turnaround time − burst time.

Example: Three processes arrive at time 0 with burst times P1 = 10, P2 = 4, P3 = 6. Time quantum = 3.

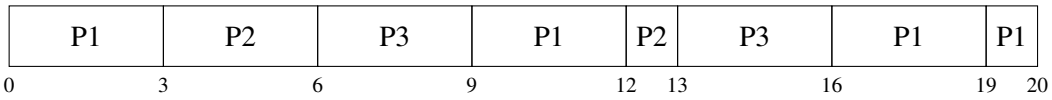


Fig: Gantt chart for Round Robin (time quantum = 3).

Process	Burst	Completion	Turnaround	Waiting
P1	10	20	20	10
P2	4	13	13	9
P3	6	16	16	10
		Average	16.33	9.67

Pgm: Write a C program to simulate Round Robin scheduling using a circular queue.

```
/* Round Robin CPU scheduling using a circular queue.
 * All processes arrive at time 0. */
#include <stdio.h>

#define MAX 10

int q[MAX], front = 0, rear = -1, count = 0;

void insert(int p)
{
    rear = (rear + 1) % MAX;
    q[rear] = p;
    count++;
}

int delete(void)
{
    int p = q[front];

    front = (front + 1) % MAX;
    count--;
    return p;
}

int main(void)
{
    int n, tq, i, p, run, time = 0;
    int burst[MAX], remain[MAX], finish[MAX];
    float total_wt = 0, total_tat = 0;

    printf("Enter the number of processes: ");
    scanf("%d", &n);
    printf("Enter burst time of each process: ");
    for (i = 0; i < n; i++) {
        scanf("%d", &burst[i]);
        remain[i] = burst[i];
        insert(i);                    /* all wait in the queue */
    }
    printf("Enter the time quantum: ");
```

```
    scanf("%d", &tq);

    printf("\nExecution order:\n");
    while (count > 0) {
        p = delete();                   /* take the front process */
        run = (remain[p] < tq) ? remain[p] : tq;
        printf("%d-%d: P%d\n", time, time + run, p + 1);
        time = time + run;
        remain[p] = remain[p] - run;
        if (remain[p] > 0)
            insert(p);           /* not finished: back to rear */
        else
            finish[p] = time;           /* finished */
    }

    printf("\nProcess  Burst  Turnaround  Waiting\n");
    for (i = 0; i < n; i++) {
        int tat = finish[i];            /* arrival time is 0 */
        int wt = tat - burst[i];
        printf("P%-7d %-6d %-11d %d\n",
               i + 1, burst[i], tat, wt);
        total_tat += tat;
        total_wt += wt;
    }
    printf("Average turnaround time = %.2f\n", total_tat / n);
    printf("Average waiting time    = %.2f\n", total_wt / n);
    return 0;
}
```

Sample Output:

```
$ ./round_robin
Enter the number of processes: 3
Enter burst time of each process: 10 4 6
Enter the time quantum: 3
Execution order:
0-3: P1
3-6: P2
6-9: P3
9-12: P1
12-13: P2
13-16: P3
16-19: P1
19-20: P1
Process  Burst  Turnaround  Waiting
P1       10     20          10
P2       4      13          9
P3       6      16          10
Average turnaround time = 16.33
Average waiting time    = 9.67
```

Question Bank

M: Marks, L: RBT (Revised Bloom's Taxonomy) level, C: Course Outcome.

Stack: Basics and Array Representation

Q. No.	Question	M	L	C
1	Define a stack. Why is it called a LIFO structure? Show the stack contents after each of the operations: PUSH 12, PUSH 7, PUSH 30, POP, PUSH 18, POP, POP.	06	L2	CO1
2	List the basic operations on a stack. Explain the overflow and underflow conditions of a stack implemented using an array.	06	L2	CO1
3	Write C functions for PUSH, POP and PEEK on a stack implemented with a static array. Explain why `top` is declared as a global variable.	08	L3	CO2
4	Develop a menu driven C program to perform PUSH, POP, PEEK and DISPLAY operations on a stack of integers using a static array, with proper overflow and underflow messages.	09	L3	CO2

Stack using Dynamic Array and Linked List

Q. No.	Question	M	L	C
5	What is the limitation of a stack implemented with a static array? Explain how a dynamic array removes this limitation using `malloc()` and `realloc()`.	06	L2	CO1
6	Write a C program to implement a stack using a dynamic array whose size is doubled whenever the stack becomes full.	08	L3	CO2
7	With a neat diagram, explain how a stack is represented using a linked list. Write C functions to PUSH and POP an element.	08	L3	CO2
8	Compare array representation and linked list representation of a stack.	05	L2	CO1

Multiple Stacks

Q. No.	Question	M	L	C
9	Explain how two stacks can be stored in a single array so that overflow occurs only when the array is completely full. Draw a diagram showing top1 and top2.	06	L2	CO1
10	Write a C program to implement two stacks in one array with functions push(stack_no, value), pop(stack_no) and display().	09	L3	CO2

Q. No.	Question	M	L	C
11	How can n stacks be implemented in a single array? What is the drawback of dividing the array into equal parts?	05	L2	CO1

Applications of Stack

Q. No.	Question	M	L	C
12	List the applications of a stack. Explain how a stack is used to reverse a list, with an example.	06	L2	CO1
13	Write a C program to read n numbers and print them in reverse order using a stack.	06	L3	CO2
14	Write a C program to compute the factorial of a number (i) using a stack and (ii) using recursion.	08	L3	CO2
15	What are infix, prefix and postfix expressions? Why do compilers convert infix expressions to postfix?	05	L2	CO1
16	Give the operator precedence table (stack and input precedence) and state the rules for deciding when to PUSH and when to POP during infix to postfix conversion.	06	L2	CO1
17	Convert the following infix expressions to postfix using a stack, showing the stack at each step: (i) A ∗ (B + C) / D − E (ii) (P − Q) ˆ R ∗ (S + T / U)	08	L3	CO2
18	Convert to postfix, showing the stack contents at each step: (i) (9 − 3) ∗ (2 + 4) / 6 (ii) X + Y ∗ Z ˆ 2 − W	08	L3	CO2
19	Write a C program to convert a valid infix expression to postfix using the operator precedence table.	09	L3	CO2
20	Evaluate the following postfix expressions using a stack, showing op1, op2 and the stack at each step (ˆ : power, % : modulus): (i) 4 5 + 3 2 − ∗ 6 / (ii) 9 3 / 2 ˆ 7 4 % +	08	L3	CO2
21	Write an algorithm and a C program to evaluate a postfix expression containing single-digit operands.	09	L3	CO2
22	What is recursion? Explain the base case and the recursive case. With a diagram, show how the system stack is used when fact(4) is called.	08	L2	CO1
23	State the rules of the Tower of Hanoi. Write a recursive C function to solve it and draw the recursion tree for 3 disks moved from peg A to peg C via peg B.	09	L3	CO2
24	How many moves are needed to solve the Tower of Hanoi with n disks? List all the moves for 3 disks.	05	L3	CO1

Queue: Basics and Representation

Q. No.	Question	M	L	C
25	Define a queue. Why is it called a FIFO structure? Show the queue contents with front and rear after: INSERT 8, INSERT 14, INSERT 3, DELETE, INSERT 21, DELETE.	06	L2	CO1
26	Explain the array representation of a queue. State the overflow, underflow and empty conditions.	06	L2	CO1
27	Develop a menu driven C program to perform INSERT, DELETE and DISPLAY operations on a linear queue using a static array, with suitable error messages.	09	L3	CO2
28	With a neat diagram, explain how a queue is represented using a linked list. Write C functions to insert and delete an element.	08	L3	CO2
29	Compare a stack and a queue.	04	L2	CO1

Circular Queue, Deque and Priority Queue

Q. No.	Question	M	L	C
30	What is the drawback of a linear queue? Explain how a circular queue overcomes it, with a neat diagram.	06	L2	CO1
31	Write the conditions for full and empty in a circular queue. Write C functions to insert and delete elements in a circular queue.	08	L3	CO2
32	For a circular queue of size 4 (front = rear = -1 initially), show front, rear and the array after: insert 5, insert 9, insert 2, delete, insert 7, insert 6, insert 1.	06	L3	CO2
33	Develop a C program to implement a circular queue of integers using an array.	09	L3	CO2
34	What is a deque? Explain input-restricted and output-restricted deques with diagrams.	06	L2	CO1
35	Write a C program to implement a double ended queue with insert_front, insert_rear, delete_front, delete_rear and display operations.	09	L3	CO2
36	What is a priority queue? Differentiate between ascending and descending priority queues, with examples.	06	L2	CO1
37	Write a C program to implement an ascending priority queue using an array kept in order of priority.	09	L3	CO2

Applications of Queues

Q. No.	Question	M	L	C
38	List any four applications of queues and explain any two.	06	L2	CO1
39	Explain the Round Robin scheduling algorithm. Why is a circular queue suitable for it?	06	L2	CO1
40	Four processes arrive at time 0 with burst times P1 = 5, P2 = 3, P3 = 8 and P4 = 2. Using Round Robin with time quantum 2, draw the Gantt chart and find the average waiting time and average turnaround time.	08	L3	CO2
41	Write a C program to simulate Round Robin scheduling using a queue and compute the waiting time and turnaround time of each process.	09	L3	CO2

********** E N D **********